

LAPORAN PROJECT

SISTEM PEMESANAN TIKET BIOSKOP

Laporan ini disusun untuk memenuhi tugas pada mata kuliah Algoritma & Struktur Data

Dosen pengampu: Taufik Ridwan, S.T., M.T.



Disusun Oleh:

Alma Laila	2510631250052
Salsabila Fadillah Putri	2510631250016
Salwa Khalipa	2510631250013
Savina Hayati Ramadhani	2510631250046

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG
TAHUN 2026

BAB 1

Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini membuat berbagai aktivitas pengelolaan data menjadi lebih mudah dan cepat, termasuk dalam bidang hiburan seperti bioskop. Pengelolaan data film dan tiket secara manual sering menimbulkan berbagai masalah, seperti kesalahan pencatatan, pencarian data yang lambat, serta kesulitan dalam mengelola stok tiket yang tersedia. Oleh karena itu, dibutuhkan sebuah sistem sederhana yang dapat membantu proses pengelolaan data bioskop secara lebih terstruktur dan efisien.

Projek program ini dibuat untuk mensimulasikan sistem manajemen tiket bioskop menggunakan bahasa pemrograman Java. Program dirancang agar dapat melakukan pengolahan data film dan tiket secara otomatis melalui fitur CRUD (Create, Read, Update, Delete), searching, sorting, dan statistik data. Dengan adanya fitur tersebut, pengguna dapat menambahkan data film, menampilkan data, mengubah informasi film, menghapus data, mencari film berdasarkan nama atau kategori, serta mengurutkan data berdasarkan ID maupun nama film.

Selain itu, projek ini juga bertujuan untuk menerapkan konsep dasar struktur data dan algoritma dalam pemrograman, seperti penggunaan array, linear search, binary search, bubble sort, dan selection sort. Program ini menjadi media pembelajaran untuk memahami bagaimana data disimpan, dicari, dan diurutkan dalam sebuah sistem sederhana. Penggunaan bahasa Java dipilih karena Java merupakan bahasa pemrograman yang bersifat object-oriented sehingga cocok digunakan untuk membangun aplikasi yang terstruktur dan mudah dikembangkan. Dalam projek ini juga diterapkan konsep class dan object melalui class **Tiket** yang merepresentasikan data setiap film pada bioskop. Dengan dibuatnya aplikasi sistem tiket bioskop ini, diharapkan proses pengelolaan data menjadi lebih efektif, cepat, dan terorganisir, sekaligus membantu mahasiswa memahami implementasi algoritma dan pemrograman dasar dalam kasus nyata.

1.2 Tujuan

- Mempermudah pengelolaan data film dan tiket bioskop agar lebih cepat dan terstruktur.
- Membantu pengguna melakukan operasi CRUD (Create, Read, Update, Delete) pada data film.
- Mempermudah pencarian data film berdasarkan nama, ID, atau kategori.
- Menerapkan konsep algoritma dan struktur data dalam bahasa pemrograman Java.
- Menyediakan sistem pemesanan tiket secara real-time agar proses pemesanan lebih efektif dan efisien.
- Menyediakan informasi statistik sederhana mengenai film dan ketersediaan tiket.

BAB 2

Analisis Kebutuhan

2.1 Fitur Sistem

Berdasarkan fungsionalitas kode yang dirancang, sistem ini menyediakan 12 fitur utama di luar menu keluar program, yaitu:

1. Load Data Otomatis (initData): Sistem secara otomatis mengunggah 30 data film awal ke dalam memori saat program pertama kali dijalankan.
2. Tambah Data Film (tambahData): Pengguna dapat menambahkan film baru ke dalam sistem dengan menginput ID, Nama, Kategori, dan Jumlah Tiket.
3. Tampilkan Data Film (tampilData): Menyajikan daftar seluruh film yang berstatus aktif di dalam sistem.
4. Edit Data Film (editData): Mengubah informasi Nama, Kategori, atau Jumlah Tiket berdasarkan parameter ID film yang dicari.
5. Hapus Data Film (hapusData): Melakukan penghapusan data secara *soft delete* (mengubah status keaktifan menjadi false tanpa menghapus elemen array agar indeks tetap konsisten).
6. Cari Berdasarkan Nama (linearSearchNama): Mencari film menggunakan nama dengan metode pencarian sekuensial (*Linear Search*).
7. Urutkan Berdasarkan ID (bubbleSortID): Mengurutkan data film dari ID terkecil ke terbesar (*ascending*) memakai metode *Bubble Sort*.
8. Urutkan Berdasarkan Nama (selectionSortNama): Mengurutkan data film secara alfabetis (*ascending*) menggunakan metode *Selection Sort*.
9. Cari Berdasarkan ID (binarySearchID): Mencari data film dengan metode *Binary Search* yang didahului proses pengurutan ID otomatis demi menjamin akurasi hasil.
10. Cari Berdasarkan Kategori (cariKategori): Menampilkan kelompok film yang memiliki kategori/genre yang sama.
11. Statistik Sistem (statistik): Menampilkan ringkasan jumlah total film yang aktif serta jumlah film yang tiketnya masih tersedia (stok > 0).
12. Urutkan Berdasarkan Jumlah Tiket (sortJumlahTiket): Mengurutkan film berdasarkan sisa kuota tiket paling sedikit menggunakan *Bubble Sort*.
13. Beli Tiket (beliTiket): Melakukan pengurangan stok tiket secara langsung jika jumlah pembelian valid dan stok mencukupi.

2.2 Struktur Data

Aplikasi ini menggunakan kombinasi struktur data berbasis objek dan array terstruktur:

1. Class / Custom Object (class Tiket)

Pada Java, sebuah *class* bertindak sebagai struktur data khusus (*custom data type*) yang membungkus beberapa variabel menjadi satu kesatuan. class Tiket ini berfungsi sebagai cetak biru (blueprint) atau representasi dari entitas sebuah film.

Di dalam struktur Tiket ini, terdapat tipe data penyusun (atribut) sebagai berikut:

- int id: Menyimpan angka identifikasi unik untuk setiap film.
- String namaFilm: Menyimpan judul film dalam bentuk teks.
- String kategori: Menyimpan genre atau kategori film dalam bentuk teks.
- int jumlahTiket: Menyimpan stok tiket yang tersedia dalam bentuk bilangan bulat.
- boolean aktif: Berfungsi sebagai *flag* (penanda) status data. Bernilai true jika data aktif, dan false jika data dihapus (*soft delete*).

2. Array of Objects (Tiket[] data)

Ini adalah struktur data utama penyimpan koleksi film di dalam memori program.

- Tipe Data: Objek Tiket.
- Kapasitas: Maksimal 100 elemen (indeks 0 hingga 99).
- Fungsi: Bertindak sebagai pangkalan data (*database*) sementara (*in-memory*) yang menyimpan daftar instansiasi objek Tiket secara berurutan. Karena menggunakan Array statis, kapasitasnya tidak bisa bertambah lebih dari 100 setelah program dijalankan.

3. Variabel Counter / Pointer Aktual (int n)

Meskipun berupa tipe data primitif sederhana (int), variabel n ini berfungsi sebagai komponen krusial dari struktur data array di atas.

- Fungsi: Bertindak sebagai penunjuk (*pointer*) untuk mengetahui jumlah data riil yang saat ini terisi di dalam array data.
- Cara Kerja: Setiap kali ada data baru yang ditambahkan, n akan digunakan sebagai indeks untuk menaruh data tersebut (`data[n++]`), lalu nilai n akan bertambah 1. Hal ini mencegah perulangan (*looping*) mengecek sisa elemen array yang masih kosong (bernilai *null*).

BAB3

Implementasi

3.1 Implementasi Kode

```
C:\Users\User> OneDrive\Documents> uts > J BioskopApp\java > Tiket > @ kategori

1 import java.util.Scanner;
2
3
4 class Tiket {
5     int id;
6     String namaFilm;
7     String kategori;
8     int jumlahTiket;
9     boolean aktif;
10
11     public Tiket(int id, String namaFilm,
12                 String kategori, int jumlahTiket) {
13
14         this.id = id;
15         this.namaFilm = namaFilm;
16         this.kategori = kategori;
17         this.jumlahTiket = jumlahTiket;
18         this.aktif = true;
19     }
20 }
21
22 public class BioskopApp {
23
24     static Tiket[] data = new Tiket[100];
25     static int n = 0;
26
27     static Scanner sc = new Scanner(System.in);
28
29     // ====== INIT DATA ======
30     static void initData() {
31
32         data[n++] = new Tiket(id: 112, namaFilm: "Avengers", kategori: "Action", jumlahTiket: 50);
33         data[n++] = new Tiket(id: 101, namaFilm: "Frozen", kategori: "Animation", jumlahTiket: 30);
34         data[n++] = new Tiket(id: 121, namaFilm: "Joker", kategori: "Drama", jumlahTiket: 20);
35         data[n++] = new Tiket(id: 110, namaFilm: "Batman", kategori: "Action", jumlahTiket: 40);
36         data[n++] = new Tiket(id: 127, namaFilm: "Spiderman", kategori: "Action", jumlahTiket: 60);
37
38         data[n++] = new Tiket(id: 107, namaFilm: "Moana", kategori: "Animation", jumlahTiket: 25);
39         data[n++] = new Tiket(id: 118, namaFilm: "Titanic", kategori: "Romance", jumlahTiket: 15);
40         data[n++] = new Tiket(id: 103, namaFilm: "Up", kategori: "Animation", jumlahTiket: 35);
41         data[n++] = new Tiket(id: 130, namaFilm: "Coco", kategori: "Animation", jumlahTiket: 45);
42         data[n++] = new Tiket(id: 115, namaFilm: "Inception", kategori: "Sci-Fi", jumlahTiket: 50);
43
44         data[n++] = new Tiket(id: 105, namaFilm: "Interstellar", kategori: "Sci-Fi", jumlahTiket: 20);
45         data[n++] = new Tiket(id: 125, namaFilm: "Matrix", kategori: "Sci-Fi", jumlahTiket: 30);
46         data[n++] = new Tiket(id: 109, namaFilm: "Gladiator", kategori: "Action", jumlahTiket: 25);
47         data[n++] = new Tiket(id: 120, namaFilm: "Cars", kategori: "Animation", jumlahTiket: 40);
48         data[n++] = new Tiket(id: 111, namaFilm: "Aladdin", kategori: "Fantasy", jumlahTiket: 35);
49
50         data[n++] = new Tiket(id: 129, namaFilm: "Mulan", kategori: "Action", jumlahTiket: 20);
51         data[n++] = new Tiket(id: 104, namaFilm: "Encanto", kategori: "Animation", jumlahTiket: 30);
52         data[n++] = new Tiket(id: 122, namaFilm: "Dune", kategori: "Sci-Fi", jumlahTiket: 45);
53         data[n++] = new Tiket(id: 114, namaFilm: "Thor", kategori: "Action", jumlahTiket: 50);
54         data[n++] = new Tiket(id: 126, namaFilm: "Hulk", kategori: "Action", jumlahTiket: 60);
55
56         data[n++] = new Tiket(id: 102, namaFilm: "Iron Man", kategori: "Action", jumlahTiket: 50);
57         data[n++] = new Tiket(id: 119, namaFilm: "Doctor Strange", kategori: "Fantasy", jumlahTiket: 35);
58         data[n++] = new Tiket(id: 108, namaFilm: "Black Panther", kategori: "Action", jumlahTiket: 40);
59         data[n++] = new Tiket(id: 124, namaFilm: "Soul", kategori: "Animation", jumlahTiket: 25);
60         data[n++] = new Tiket(id: 116, namaFilm: "Luca", kategori: "Animation", jumlahTiket: 20);
61
62         data[n++] = new Tiket(id: 128, namaFilm: "Turning Red", kategori: "Animation", jumlahTiket: 30);
63         data[n++] = new Tiket(id: 106, namaFilm: "Venom", kategori: "Action", jumlahTiket: 45);
64         data[n++] = new Tiket(id: 123, namaFilm: "Deadpool", kategori: "Action", jumlahTiket: 50);
65         data[n++] = new Tiket(id: 113, namaFilm: "Minions", kategori: "Animation", jumlahTiket: 60);
66         data[n++] = new Tiket(id: 117, namaFilm: "Shrek", kategori: "Animation", jumlahTiket: 55);
67
68
69         data[n++] = new Tiket(id: 135, namaFilm: "Barbie", kategori: "Comedy", jumlahTiket: 40);
70         data[n++] = new Tiket(id: 131, namaFilm: "Oppenheimer", kategori: "Drama", jumlahTiket: 35);
71         data[n++] = new Tiket(id: 140, namaFilm: "John Wick", kategori: "Action", jumlahTiket: 50);
72         data[n++] = new Tiket(id: 132, namaFilm: "Elemental", kategori: "Animation", jumlahTiket: 20);
73         data[n++] = new Tiket(id: 138, namaFilm: "The Nun", kategori: "Horror", jumlahTiket: 25);
74
75         data[n++] = new Tiket(id: 134, namaFilm: "Insidious", kategori: "Horror", jumlahTiket: 30);
76         data[n++] = new Tiket(id: 136, namaFilm: "Fast X", kategori: "Action", jumlahTiket: 45);
77         data[n++] = new Tiket(id: 133, namaFilm: "Mish", kategori: "Animation", jumlahTiket: 15);
78         data[n++] = new Tiket(id: 139, namaFilm: "Aquaman", kategori: "Action", jumlahTiket: 55);
79         data[n++] = new Tiket(id: 137, namaFilm: "Kung Fu Panda", kategori: "Animation", jumlahTiket: 40);
80
81         data[n++] = new Tiket(id: 145, namaFilm: "Toy Story", kategori: "Animation", jumlahTiket: 35);
82         data[n++] = new Tiket(id: 141, namaFilm: "Avatar", kategori: "Sci-Fi", jumlahTiket: 60);
83         data[n++] = new Tiket(id: 150, namaFilm: "Conjuring", kategori: "Horror", jumlahTiket: 20);
84         data[n++] = new Tiket(id: 142, namaFilm: "Transformers", kategori: "Action", jumlahTiket: 45);
85         data[n++] = new Tiket(id: 148, namaFilm: "Inside Out", kategori: "Animation", jumlahTiket: 50);
86
87         data[n++] = new Tiket(id: 144, namaFilm: "Annabelle", kategori: "Horror", jumlahTiket: 25);
88         data[n++] = new Tiket(id: 146, namaFilm: "Top Gun", kategori: "Action", jumlahTiket: 30);
89         data[n++] = new Tiket(id: 143, namaFilm: "Cinderella", kategori: "Fantasy", jumlahTiket: 15);
90         data[n++] = new Tiket(id: 149, namaFilm: "Mission Impossible", kategori: "Action", jumlahTiket: 55);
91         data[n++] = new Tiket(id: 147, namaFilm: "How to Train Your Dragon", kategori: "Animation", jumlahTiket: 40);
92
93         System.out.println(x: ">> 80 data berhasil di-load!");
94     }
95
96     // ====== TAMBAH DATA ======
97     static void tambahData() {
98
99         try {
100
101             System.out.print(x: "Masukkan ID: ");
102             int id = Integer.parseInt(sc.nextLine());
103
104             System.out.print(x: "Masukkan Nama Film: ");
105             String nama = sc.nextLine();
106
107         } catch (Exception e) {
108             System.out.println("Error: " + e.getMessage());
109         }
110     }
111 }
```

C:\Users\User> OneDrive > Documents > uts > J BioskopApp1.java > Tiket > kategori

```
22 public class BioskopApp1 {
97     static void tambahData() {
107         System.out.print(s: "Masukkan Kategori: ");
108         String kategori = sc.nextLine();
109
110         System.out.print(s: "Masukkan Jumlah Tiket: ");
111         int jumlah = Integer.parseInt(sc.nextLine());
112
113         data[n++] = new Tiket(id, nama, kategori, jumlah);
114
115         System.out.println(k: ">>> Data berhasil ditambahkan!");
116
117     } catch (Exception e) {
118
119         System.out.println(k: ">>> Input tidak valid!");
120     }
121 }
122
123 // ===== TAMPIL DATA =====
124 static void tampilData() {
125
126     System.out.println(k: "\n=== DATA FILM ===");
127
128     for (int i = 0; i < n; i++) {
129
130         if (data[i].aktif) {
131
132             System.out.println(
133                 data[i].id + " | " +
134                 data[i].namaFilm + " | " +
135                 data[i].kategori + " | " +
136                 data[i].jumlahTiket
137             );
138         }
139     }
140 }
141
142 // ===== EDIT DATA =====
143 static void editData() {
144
145     try {
146
147         System.out.print(s: "Masukkan ID Film: ");
148         int id = Integer.parseInt(sc.nextLine());
149
150         for (int i = 0; i < n; i++) {
151
152             if (data[i].id == id && data[i].aktif) {
153
154                 System.out.print(s: "Nama Baru: ");
155                 data[i].namaFilm = sc.nextLine();
156
157                 System.out.print(s: "Kategori Baru: ");
158                 data[i].kategori = sc.nextLine();

```

C:\Users\User> OneDrive > Documents > uts > J BioskopApp1.java > Tiket > kategori

```
22 public class BioskopApp1 {
143     static void editData() {
160         System.out.print(s: "Jumlah Tiket Baru: ");
161         data[i].jumlahTiket =
162             Integer.parseInt(sc.nextLine());
163
164         System.out.println(k: ">>> Data berhasil diupdate!");
165         return;
166     }
167 }
168
169     System.out.println(k: ">>> Data tidak ditemukan!");
170
171 } catch (Exception e) {
172
173     System.out.println(k: ">>> Input tidak valid!");
174 }
175 }
176
177 // ===== HAPUS DATA =====
178 static void hapusData() {
179
180     try {
181
182         System.out.print(s: "Masukkan ID Film: ");
183         int id = Integer.parseInt(sc.nextLine());
184
185         for (int i = 0; i < n; i++) {
186
187             if (data[i].id == id && data[i].aktif) {
188
189                 data[i].aktif = false;
190
191                 System.out.println(k: ">>> Data berhasil dihapus!");
192                 return;
193             }
194         }
195
196         System.out.println(k: ">>> Data tidak ditemukan!");
197
198     } catch (Exception e) {
199
200         System.out.println(k: ">>> Input tidak valid!");
201     }
202 }
203
204 // ===== LINEAR SEARCH =====
205 static void linearSearchNama() {
206
207     System.out.print(s: "Cari Nama Film: ");
208     String cari = sc.nextLine();
209
210     boolean ketemu = false;
211 }

```

```

C:\Users\User> OneDrive > Documents > uts > J BioskopApp1.java > Tiket > kategori
22 public class BioskopApp1 {
285 static void linearSearchNama() {
211
212     for (int i = 0; i < n; i++) {
213
214         if (data[i].aktif &&
215             data[i].namaFilm.equalsIgnoreCase(cari)) {
216
217             System.out.println(
218                 ">> Ditemukan: " +
219                 data[i].namaFilm
220             );
221
222             ketemu = true;
223         }
224     }
225
226     if (!ketemu) {
227
228         System.out.println(x: ">> Film tidak ditemukan!");
229     }
230 }
231
232 // ===== BUBBLE SORT ID =====
233 static void bubbleSortID() {
234
235     for (int i = 0; i < n - 1; i++) {
236
237         for (int j = 0; j < n - i - 1; j++) {
238
239             if (data[j].id > data[j + 1].id) {
240
241                 Tiket temp = data[j];
242                 data[j] = data[j + 1];
243                 data[j + 1] = temp;
244             }
245         }
246     }
247
248     System.out.println(x: ">> Data berhasil diurutkan berdasarkan ID!");
249
250     tampilData();
251 }
252
253 // ===== SELECTION SORT NAMA =====
254 static void selectionSortNama() {
255
256     for (int i = 0; i < n - 1; i++) {
257
258         int min = i;
259
260         for (int j = i + 1; j < n; j++) {
261

```

```

C:\Users\User> OneDrive > Documents > uts > J BioskopApp1.java > Tiket > kategori
22 public class BioskopApp1 {
254 static void selectionSortNama() {
262
263         if (data[j].namaFilm.compareToIgnoreCase(
264             data[min].namaFilm) < 0) {
265
266             min = j;
267         }
268
269         Tiket temp = data[i];
270         data[i] = data[min];
271         data[min] = temp;
272     }
273
274     System.out.println(x: ">> Data berhasil diurutkan berdasarkan Nama!");
275
276     tampilData();
277 }
278
279 // ===== SORT JUMLAH TIKET =====
280 static void sortJumlahTiket() {
281
282     for (int i = 0; i < n - 1; i++) {
283
284         for (int j = 0; j < n - i - 1; j++) {
285
286             if (data[j].jumlahTiket >
287                 data[j + 1].jumlahTiket) {
288
289                 Tiket temp = data[j];
290                 data[j] = data[j + 1];
291                 data[j + 1] = temp;
292             }
293         }
294     }
295
296     System.out.println(
297         x: ">> Data berhasil diurutkan berdasarkan jumlah tiket!"
298     );
299
300     tampilData();
301 }
302
303 // ===== BINARY SEARCH =====
304 static int binarySearchID(int id) {
305
306     int left = 0;
307     int right = n - 1;
308
309     while (left <= right) {
310
311         int mid = (left + right) / 2;
312

```

C:\Users\User> OneDrive\Documents> uts> J BioskopApp1.java> Tiket> kategori

```
22 public class BioskopApp1 {
304 static int binarySearchID(int id) {
313     if (data[mid].id == id) {
314         return mid;
315     }
316 }
317
318     else if (data[mid].id < id) {
319         left = mid + 1;
320     }
321 }
322
323     else {
324         right = mid - 1;
325     }
326 }
327
328     return -1;
329 }
330
331 // ===== CARI KATEGORI =====
332 static void cariKategori() {
333     System.out.print("Masukkan Kategori: ");
334     String kategori = sc.nextLine();
335
336     boolean ketemu = false;
337
338     for (int i = 0; i < n; i++) {
339         if (data[i].aktif &&
340             data[i].kategori.equalsIgnoreCase(kategori)) {
341             System.out.println(
342                 data[i].id + " | " +
343                 data[i].namaFilm + " | " +
344                 data[i].kategori + " | " +
345                 data[i].jumlahTiket
346             );
347             ketemu = true;
348         }
349     }
350
351     if (!ketemu) {
352         System.out.println("X: >> Kategori tidak ditemukan!");
353     }
354 }
355
356 // ===== BELI TIKET =====
357 static void beliTiket() {
```

C:\Users\User> OneDrive\Documents> uts> J BioskopApp1.java> Tiket> kategori

```
22 public class BioskopApp1 {
363 static void beliTiket() {
365     try {
366         System.out.print("Masukkan ID Film: ");
367         int id = Integer.parseInt(sc.nextLine());
368
369         boolean ketemu = false;
370
371         for (int i = 0; i < n; i++) {
372             if (data[i].id == id && data[i].aktif) {
373                 ketemu = true;
374
375                 System.out.println(
376                     "Film: " + data[i].namaFilm
377                 );
378
379                 System.out.println(
380                     "Stok Tiket: " +
381                     data[i].jumlahTiket
382                 );
383
384                 System.out.print(
385                     "Jumlah tiket dibeli: "
386                 );
387
388                 int beli =
389                     Integer.parseInt(sc.nextLine());
390
391                 // Validasi jumlah tiket
392                 if (beli <= 0) {
393                     System.out.println(
394                         "X: >> Jumlah tiket tidak valid!"
395                     );
396                 }
397
398                 else if (beli >
399                     data[i].jumlahTiket) {
400                     System.out.println(
401                         "X: >> Stok tiket tidak mencukupi!"
402                     );
403                 }
404
405                 else {
406                     data[i].jumlahTiket -= beli;
407                     System.out.println(
408                         "X: >> Tiket berhasil dibeli!"
409                     );
410                 }
411             }
412         }
413     } catch (Exception e) {
414         System.out.println("X: >> Error!");
415     }
416 }
```


C:\Users\User > OneDrive > Documents > uts > J BioskopApp1.java > Tiket > kategori

```
22 public class BioskopApp1 {
363     static void beliTiket() {
416     }
417     System.out.println(
418         ">> Sisa tiket: " +
419         data[i].jumlahTiket
420     );
421     }
422     }
423     return;
424     }
425     }
426     }
427     if (!ketemu) {
428         System.out.println(
429             k: ">> Film tidak ditemukan!"
430         );
431     }
432     }
433     }
434     } catch (Exception e) {
435     }
436     System.out.println(k: ">> Input tidak valid!");
437     }
438     }
439     }
440     // ===== STATISTIK =====
441     static void statistik() {
442     }
443     int total = 0;
444     int tersedia = 0;
445     for (int i = 0; i < n; i++) {
446     }
447     if (data[i].aktif) {
448     }
449     total++;
450     if (data[i].jumlahTiket > 0) {
451     }
452     tersedia++;
453     }
454     }
455     }
456     }
457     }
458     }
459     System.out.println(k: "\n=== STATISTIK ===");
460     System.out.println("Total Film Aktif : " + total);
461     System.out.println("Film Tersedia : " + tersedia);
462     }
463     }
464     // ===== MAIN =====
465     Run | Debug
466     public static void main(String[] args) {
```

C:\Users\User > OneDrive > Documents > uts > J BioskopApp1.java > Tiket > kategori

```
22 public class BioskopApp1 {
466     public static void main(String[] args) {
467     }
468     initData();
469     int pilih = -1;
470     do {
471     }
472     }
473     System.out.println(k: "\n=== MENU BIOSKOP ===");
474     System.out.println(k: "1. Tambah Data");
475     System.out.println(k: "2. Tampil Data");
476     System.out.println(k: "3. Edit Data");
477     System.out.println(k: "4. Hapus Data");
478     System.out.println(k: "5. Cari Nama");
479     System.out.println(k: "6. Urutkan ID");
480     System.out.println(k: "7. Urutkan Nama");
481     System.out.println(k: "8. Cari ID");
482     System.out.println(k: "9. Cari Kategori");
483     System.out.println(k: "10. Statistik");
484     System.out.println(k: "11. Urutkan Jumlah Tiket");
485     System.out.println(k: "12. Beli Tiket");
486     System.out.println(k: "0. Keluar");
487     }
488     System.out.print(s: "Pilih Menu: ");
489     try {
490     }
491     } catch (Exception e) {
492     }
493     System.out.println(k: ">> Input harus angka!");
494     continue;
495     }
496     }
497     switch (pilih) {
498     }
499     }
500     case 1:
501     tambahData();
502     break;
503     case 2:
504     tampilData();
505     break;
506     case 3:
507     editData();
508     break;
509     case 4:
510     hapusData();
511     break;
512     case 5:
513     cariNama();
514     break;
515     case 6:
516     urutkanID();
517     break;
518     case 7:
519     urutkanNama();
520     break;
521     case 8:
522     cariID();
523     break;
524     case 9:
525     cariKategori();
526     break;
527     case 10:
528     statistik();
529     break;
530     case 11:
531     urutkanJumlahTiket();
532     break;
533     case 12:
534     beliTiket();
535     break;
536     case 0:
537     keluar();
538     break;
539     }
540     }
541     }
542     }
543     }
544     }
545     }
546     }
547     }
548     }
549     }
550     }
551     }
552     }
553     }
554     }
555     }
556     }
557     }
558     }
559     }
560     }
561     }
562     }
563     }
564     }
565     }
566     }
567     }
568     }
569     }
570     }
571     }
572     }
573     }
574     }
575     }
576     }
577     }
578     }
579     }
580     }
581     }
582     }
583     }
584     }
585     }
586     }
587     }
588     }
589     }
590     }
591     }
592     }
593     }
594     }
595     }
596     }
597     }
598     }
599     }
600     }
601     }
602     }
603     }
604     }
605     }
606     }
607     }
608     }
609     }
610     }
611     }
612     }
613     }
614     }
615     }
616     }
617     }
618     }
619     }
620     }
621     }
622     }
623     }
624     }
625     }
626     }
627     }
628     }
629     }
630     }
631     }
632     }
633     }
634     }
635     }
636     }
637     }
638     }
639     }
640     }
641     }
642     }
643     }
644     }
645     }
646     }
647     }
648     }
649     }
650     }
651     }
652     }
653     }
654     }
655     }
656     }
657     }
658     }
659     }
660     }
661     }
662     }
663     }
664     }
665     }
666     }
667     }
668     }
669     }
670     }
671     }
672     }
673     }
674     }
675     }
676     }
677     }
678     }
679     }
680     }
681     }
682     }
683     }
684     }
685     }
686     }
687     }
688     }
689     }
690     }
691     }
692     }
693     }
694     }
695     }
696     }
697     }
698     }
699     }
700     }
701     }
702     }
703     }
704     }
705     }
706     }
707     }
708     }
709     }
710     }
711     }
712     }
713     }
714     }
715     }
716     }
717     }
718     }
719     }
720     }
721     }
722     }
723     }
724     }
725     }
726     }
727     }
728     }
729     }
730     }
731     }
732     }
733     }
734     }
735     }
736     }
737     }
738     }
739     }
740     }
741     }
742     }
743     }
744     }
745     }
746     }
747     }
748     }
749     }
750     }
751     }
752     }
753     }
754     }
755     }
756     }
757     }
758     }
759     }
760     }
761     }
762     }
763     }
764     }
765     }
766     }
767     }
768     }
769     }
770     }
771     }
772     }
773     }
774     }
775     }
776     }
777     }
778     }
779     }
780     }
781     }
782     }
783     }
784     }
785     }
786     }
787     }
788     }
789     }
790     }
791     }
792     }
793     }
794     }
795     }
796     }
797     }
798     }
799     }
800     }
801     }
802     }
803     }
804     }
805     }
806     }
807     }
808     }
809     }
810     }
811     }
812     }
813     }
814     }
815     }
816     }
817     }
818     }
819     }
820     }
821     }
822     }
823     }
824     }
825     }
826     }
827     }
828     }
829     }
830     }
831     }
832     }
833     }
834     }
835     }
836     }
837     }
838     }
839     }
840     }
841     }
842     }
843     }
844     }
845     }
846     }
847     }
848     }
849     }
850     }
851     }
852     }
853     }
854     }
855     }
856     }
857     }
858     }
859     }
860     }
861     }
862     }
863     }
864     }
865     }
866     }
867     }
868     }
869     }
870     }
871     }
872     }
873     }
874     }
875     }
876     }
877     }
878     }
879     }
880     }
881     }
882     }
883     }
884     }
885     }
886     }
887     }
888     }
889     }
890     }
891     }
892     }
893     }
894     }
895     }
896     }
897     }
898     }
899     }
900     }
901     }
902     }
903     }
904     }
905     }
906     }
907     }
908     }
909     }
910     }
911     }
912     }
913     }
914     }
915     }
916     }
917     }
918     }
919     }
920     }
921     }
922     }
923     }
924     }
925     }
926     }
927     }
928     }
929     }
930     }
931     }
932     }
933     }
934     }
935     }
936     }
937     }
938     }
939     }
940     }
941     }
942     }
943     }
944     }
945     }
946     }
947     }
948     }
949     }
950     }
951     }
952     }
953     }
954     }
955     }
956     }
957     }
958     }
959     }
960     }
961     }
962     }
963     }
964     }
965     }
966     }
967     }
968     }
969     }
970     }
971     }
972     }
973     }
974     }
975     }
976     }
977     }
978     }
979     }
980     }
981     }
982     }
983     }
984     }
985     }
986     }
987     }
988     }
989     }
990     }
991     }
992     }
993     }
994     }
995     }
996     }
997     }
998     }
999     }
1000    }
```

```

C: > Users > User > OneDrive > Documents > uts > J BioskopApp1.java > Tiket > kategori
22 public class BioskopApp1 {
466 public static void main(String[] args) {

518
519
520     case 5:
521         linearSearchNama();
522         break;
523
524     case 6:
525         bubbleSortID();
526         break;
527
528     case 7:
529         selectionSortNama();
530         break;
531
532     case 8:
533         bubbleSortID();
534
535         try {
536
537             System.out.print(
538                 "Masukkan ID yang dicari: "
539             );
540
541             int cariID =
542                 Integer.parseInt(
543                     sc.nextLine()
544                 );
545
546             int hasil =
547                 binarySearchID(cariID);
548
549             if (hasil != -1 &&
550                 data[hasil].aktif) {
551
552                 System.out.println(
553                     ">> Film ditemukan: " +
554                     data[hasil].namaFilm
555                 );
556
557             } else {
558
559                 System.out.println(
560                     "X: >> Data tidak ditemukan!"
561                 );
562             }
563
564         } catch (Exception e) {
565
566             System.out.println(
567                 "X: >> Input tidak valid!"
568             );

```

```

C: > Users > User > OneDrive > Documents > uts > J BioskopApp1.java > Tiket > kategori
22 public class BioskopApp1 {
466 public static void main(String[] args) {

569
570
571         break;
572
573     case 9:
574         cariKategori();
575         break;
576
577     case 10:
578         statistik();
579         break;
580
581     case 11:
582         sortJumlahTiket();
583         break;
584
585     case 12:
586         beliTiket();
587         break;
588
589     case 0:
590         System.out.println("Program selesai.");
591         break;
592
593     default:
594         System.out.println(
595             "X: >> Menu tidak tersedia!"
596         );
597     }
598
599 } while (pilih != 0);
600 }
601 }

```

3.2 Penjelasan Alur Kode

- Inisialisasi: Saat fungsi main dipanggil, method initData() dijalankan terlebih dahulu untuk mengisi array data dengan 30 instansiasi objek Tiket. Variabel counter n bertambah seiring bertambahnya data.
- Looping Menu Utama: Menggunakan struktur do-while untuk menjaga aplikasi tetap berjalan sampai pengguna memilih angka 0. Blok try-catch membungkus parsing input menu demi mencegah aplikasi *crash* akibat kesalahan ketik huruf.
- Mekanisme CRUD: Operasi Tambah, Tampil, Edit, dan Hapus memanfaatkan perulangan for hingga batas counter n. Fitur hapus data diimplementasikan via enkapsulasi logika (data[i].aktif = false), sehingga data lama tidak bergeser melainkan diabaikan saat fungsi cetak dipanggil.
- Logika Transaksi: Fungsi beliTiket() mencari kecocokan ID film, menampilkan kapasitas yang tersisa, kemudian mengurangi variabel jumlahTiket dari objek target setelah lolos uji validasi kuantitas.

BAB 4

Analisis Kompleksitas

4.1 Tabel Kompleksitas

Fitur	Algoritma	Kompleksitas
Tambah Data	Insert Array	$O(1)$
Tampil Data	Traversal	$O(n)$
Edit Data	Linear Search	$O(n)$
Hapus Data	Linear Search	$O(n)$
Cari Nama Film	Linear Search	$O(n)$
Cari Kategori	Linear Search	$O(n)$
Cari ID Film	Binary Search	$O(\log n)$
Sorting ID	Bubble Sort	$O(n^2)$
Sorting Nama	Selection Sort	$O(n^2)$
Sorting Jumlah Tiket	Bubble Sort	$O(n^2)$
Beli Tiket	Linear Search	$O(n)$
Statistik Data	Traversal	$O(n)$

4.2 Penjelasan Analisis

Berdasarkan tabel kompleksitas pada Tabel 4.1, setiap fitur dalam sistem memiliki tingkat efisiensi algoritma yang berbeda-beda tergantung proses yang dilakukan.

1. Penambahan Data ($O(1)$)

Proses penambahan data film memiliki kompleksitas waktu $O(1)$ karena data baru langsung disimpan pada indeks terakhir array melalui satu kali eksekusi tanpa perlu melakukan perulangan. Hal ini membuat proses penambahan data berjalan sangat cepat dan stabil karena tidak dipengaruhi oleh banyaknya jumlah data yang tersimpan di dalam sistem.

2. Pencarian Sekuensial / Linear Search ($O(n)$)

Fitur seperti tampil data, edit data, hapus data, pencarian nama film, pencarian kategori, pembelian tiket, dan statistik menggunakan metode traversal atau Linear Search. Algoritma ini bekerja dengan cara menelusuri isi array satu per satu mulai dari indeks

awal hingga data ditemukan atau mencapai akhir array. Oleh karena itu, jika jumlah data meningkat sebesar n , maka waktu proses juga akan meningkat secara linear mengikuti jumlah data tersebut.

3. Algoritma Pengurutan / Sorting ($O(n^2)$)

Proses pengurutan data menggunakan algoritma Bubble Sort dan Selection Sort yang memiliki kompleksitas $O(n^2)$. Kedua algoritma tersebut menggunakan perulangan bersarang (nested loop), di mana setiap data akan dibandingkan dengan data lainnya secara berulang untuk menentukan posisi yang sesuai. Pada kondisi terburuk, sistem harus melakukan perbandingan sebanyak $n \times n$, sehingga proses sorting menjadi kurang efisien apabila jumlah data sangat besar.

4. Pencarian Biner / Binary Search ($O(\log n)$)

Fitur pencarian berdasarkan ID film menggunakan algoritma Binary Search dengan kompleksitas $O(\log n)$. Algoritma ini bekerja dengan membagi ruang pencarian menjadi dua bagian pada setiap proses iterasi sehingga jumlah data yang diperiksa terus berkurang secara signifikan. Dibandingkan Linear Search, metode ini jauh lebih cepat dan efisien untuk data berukuran besar. Namun, agar Binary Search dapat digunakan, data harus terlebih dahulu diurutkan berdasarkan ID menggunakan Bubble Sort.

BAB 5

Pengujian

Berikut merupakan representasi tekstual dari *screenshot* luaran konsol (*Console Output*) sistem pada berbagai skenario operasional:

1. Verifikasi Load Data & Tampil Data:

Pastikan program memuat 30 data awal, lalu tampilkan daftarnya.

- Input: Ketik 2 pada Menu Bioskop lalu tekan Enter.
- Ekspektasi Output: Muncul tulisan === DATA FILM === diikuti daftar 30 film (mulai dari 112 | Avengers hingga 117 | Shrek).

```
=== MENU BIOSKOP ===
1. Tambah Data
2. Tampil Data
3. Edit Data
4. Hapus Data
5. Cari Nama
6. Urutkan ID
7. Urutkan Nama
8. Cari ID
9. Cari Kategori
10. Statistik
11. Urutkan Jumlah Tiket
12. Beli Tiket
0. Keluar
Pilih Menu: 2

=== DATA FILM ===
112 | Avengers | Action | 50
101 | Frozen | Animation | 30
121 | Joker | Drama | 20
110 | Batman | Action | 40
127 | Spiderman | Action | 60
107 | Moana | Animation | 25
118 | Titanic | Romance | 15
103 | Up | Animation | 35
130 | Coco | Animation | 45
115 | Inception | Sci-Fi | 50
105 | Interstellar | Sci-Fi | 20
125 | Matrix | Sci-Fi | 30
109 | Gladiator | Action | 25
120 | Cars | Animation | 40
111 | Aladdin | Fantasy | 35
129 | Mulan | Action | 20
104 | Encanto | Animation | 30
122 | Dune | Sci-Fi | 45
114 | Thor | Action | 50
126 | Hulk | Action | 60
102 | Iron Man | Action | 50
119 | Doctor Strange | Fantasy | 35
108 | Black Panther | Action | 40
124 | Soul | Animation | 25
116 | Luca | Animation | 20
128 | Turning Red | Animation | 30
106 | Venom | Action | 45
123 | Deadpool | Action | 50
113 | Minions | Animation | 60
```

2. Menguji Fitur Tambah Data:

Tambahkan satu film baru ke dalam sistem.

- Input: Ketik 1 (Tambah Data).
- Masukkan ID: 999
- Masukkan Nama Film: Avatar 2
- Masukkan Kategori: Sci-Fi
- Masukkan Jumlah Tiket: 100
- Ekspektasi Output: >> Data berhasil ditambahkan!

```
=== MENU BIOSKOP ===
```

- ```
1. Tambah Data
2. Tampil Data
3. Edit Data
4. Hapus Data
5. Cari Nama
6. Urutkan ID
7. Urutkan Nama
8. Cari ID
9. Cari Kategori
10. Statistik
11. Urutkan Jumlah Tiket
12. Beli Tiket
0. Keluar
```

```
Pilih Menu: 1
```

```
Masukkan ID: 999
```

```
Masukkan Nama Film: Avatar 2
```

```
Masukkan Kategori: Sci-Fi
```

```
Masukkan Jumlah Tiket: 100
```

```
>> Data berhasil ditambahkan!
```

### 3. Menguji Fitur Edit Data:

Ubah data film "Avatar 2" yang baru saja ditambahkan.

- Input: Ketik 3 (Edit Data).
- Masukkan ID Film: 999
- Nama Baru: Avatar: The Way of Water
- Kategori Baru: Sci-Fi
- Jumlah Tiket Baru: 150
- Ekspektasi Output: >> Data berhasil diupdate!

```
=== MENU BIOSKOP ===
```

1. Tambah Data
2. Tampil Data
3. Edit Data
4. Hapus Data
5. Cari Nama
6. Urutkan ID
7. Urutkan Nama
8. Cari ID
9. Cari Kategori
10. Statistik
11. Urutkan Jumlah Tiket
12. Beli Tiket
0. Keluar

```
Pilih Menu: 3
```

```
Masukkan ID Film: 999
```

```
Nama Baru: The Way of Water
```

```
Kategori Baru: Sci-Fi
```

```
Jumlah Tiket Baru: 150
```

```
>> Data berhasil diupdate!
```



4. Menguji Fitur Transaksi (Beli Tiket Sukses & Gagal):  
Uji pengurangan stok tiket dan validasi jika tiket tidak cukup.  
a. Pengujian Berhasil:

- Input: Ketik 12 (Beli Tiket).
- Masukkan ID Film: 112 (Avengers).
- Jumlah tiket dibeli: 5
- Ekspektasi Output: >> Tiket berhasil dibeli! >> Sisa tiket: 45

```
=== MENU BIOSKOP ===
1. Tambah Data
2. Tampil Data
3. Edit Data
4. Hapus Data
5. Cari Nama
6. Urutkan ID
7. Urutkan Nama
8. Cari ID
9. Cari Kategori
10. Statistik
11. Urutkan Jumlah Tiket
12. Beli Tiket
0. Keluar
Pilih Menu: 12
Masukkan ID Film: 112
Film: Avengers
Stok Tiket: 50
Jumlah tiket dibeli: 5
>> Tiket berhasil dibeli!
>> Sisa tiket: 45
```

b. Pengujian Gagal (Stok Kurang):

- Input: Ketik 12 lagi.
- Masukkan ID Film: 112
- Jumlah tiket dibeli: 100
- Ekspektasi Output: >> Stok tiket tidak mencukupi!

```
=== MENU BIOSKOP ===
1. Tambah Data
2. Tampil Data
3. Edit Data
4. Hapus Data
5. Cari Nama
6. Urutkan ID
7. Urutkan Nama
8. Cari ID
9. Cari Kategori
10. Statistik
11. Urutkan Jumlah Tiket
12. Beli Tiket
0. Keluar
Pilih Menu: 12
Masukkan ID Film: 112
Film: Avengers
Stok Tiket: 45
Jumlah tiket dibeli: 100
>> Stok tiket tidak mencukupi!
```

5. Menguji Fitur Sorting (Pengurutan):

Uji apakah algoritma Bubble Sort atau Selection Sort bekerja mengatur ulang array.

- Input: Ketik 6 (Urutkan ID).
- Ekspektasi Output: Daftar film akan dicetak ulang, namun kali ini dimulai dari ID paling kecil (101 | Frozen) ke paling besar.
- Input: Ketik 11 (Urutkan Jumlah Tiket).
- Ekspektasi Output: Daftar film dicetak berdasarkan sisa tiket dari yang paling sedikit ke paling banyak.

=== MENU BIOSKOP ===

1. Tambah Data
2. Tampil Data
3. Edit Data
4. Hapus Data
5. Cari Nama
6. Urutkan ID
7. Urutkan Nama
8. Cari ID
9. Cari Kategori
10. Statistik
11. Urutkan Jumlah Tiket
12. Beli Tiket
0. Keluar

Pilih Menu: 6

>> Data berhasil diurutkan berdasarkan ID!

=== DATA FILM ===

|     |                |           |    |
|-----|----------------|-----------|----|
| 101 | Frozen         | Animation | 30 |
| 102 | Iron Man       | Action    | 50 |
| 103 | Up             | Animation | 35 |
| 104 | Encanto        | Animation | 30 |
| 105 | Interstellar   | Sci-Fi    | 20 |
| 106 | Venom          | Action    | 45 |
| 107 | Moana          | Animation | 25 |
| 108 | Black Panther  | Action    | 40 |
| 109 | Gladiator      | Action    | 25 |
| 110 | Batman         | Action    | 40 |
| 111 | Aladdin        | Fantasy   | 35 |
| 112 | Avengers       | Action    | 45 |
| 113 | Minions        | Animation | 60 |
| 114 | Thor           | Action    | 50 |
| 115 | Inception      | Sci-Fi    | 50 |
| 116 | Luca           | Animation | 20 |
| 117 | Shrek          | Animation | 55 |
| 118 | Titanic        | Romance   | 15 |
| 119 | Doctor Strange | Fantasy   | 35 |
| 120 | Cars           | Animation | 40 |
| 121 | Joker          | Drama     | 20 |
| 122 | Dune           | Sci-Fi    | 45 |
| 123 | Deadpool       | Action    | 50 |
| 124 | Soul           | Animation | 25 |
| 125 | Matrix         | Sci-Fi    | 30 |
| 126 | Hulk           | Action    | 60 |
| 127 | Spiderman      | Action    | 60 |
| 128 | Turning Red    | Animation | 30 |
| 129 | Mulan          | Action    | 20 |
| 130 | Coco           | Animation | 45 |
| 131 | Oppenheimer    | Drama     | 35 |
| 132 | Elemental      | Animation | 20 |
| 133 | Wish           | Animation | 15 |
| 134 | Insidious      | Horror    | 30 |
| 135 | Barbie         | Comedy    | 40 |
| 136 | Fast X         | Action    | 45 |
| 137 | Kung Fu Panda  | Animation | 40 |
| 138 | The Nun        | Horror    | 25 |
| 139 | Aquaman        | Action    | 55 |
| 140 | John Wick      | Action    | 50 |
| 141 | Avatar         | Sci-Fi    | 60 |
| 142 | Transformers   | Action    | 45 |
| 143 | Cinderella     | Fantasy   | 15 |
| 144 | Annabelle      | Horror    | 25 |
| 145 | Toy Story      | Animation | 35 |
| 146 | Top Gun        | Action    | 30 |

6. Menguji Fitur Searching (Pencarian Biner):

Cari film menggunakan ID (Binary Search mewajibkan data diurutkan dulu, dan sistem Anda sudah menanganinya).

- Input: Ketik 8 (Cari ID).
- Masukkan ID yang dicari: 121
- Ekspektasi Output: >> Film ditemukan: Joker

```
=== MENU BIOSKOP ===
```

```
1. Tambah Data
2. Tampil Data
3. Edit Data
4. Hapus Data
5. Cari Nama
6. Urutkan ID
7. Urutkan Nama
8. Cari ID
9. Cari Kategori
10. Statistik
11. Urutkan Jumlah Tiket
12. Beli Tiket
0. Keluar
```

```
Pilih Menu: 8
```

```
>> Data berhasil diurutkan berdasarkan ID!
```

```
132 | Elemental | Animation | 20
133 | Wish | Animation | 15
134 | Insidious | Horror | 30
135 | Barbie | Comedy | 40
136 | Fast X | Action | 45
137 | Kung Fu Panda | Animation | 40
138 | The Nun | Horror | 25
139 | Aquaman | Action | 55
140 | John Wick | Action | 50
141 | Avatar | Sci-Fi | 60
142 | Transformers | Action | 45
143 | Cinderella | Fantasy | 15
144 | Annabelle | Horror | 25
145 | Toy Story | Animation | 35
146 | Top Gun | Action | 30
147 | How to Train Your Dragon | Animation | 40
148 | Inside Out | Animation | 50
149 | Mission Impossible | Action | 55
150 | Conjuring | Horror | 20
999 | The Way of Water | Sci-Fi | 150
Masukkan ID yang dicari: 121
>> Film ditemukan: Joker
```

7. Menguji Fitur Hapus Data (Soft Delete):

Hapus data film Avatar (ID 999) yang tadi ditambahkan.

- Input: Ketik 4 (Hapus Data).
- Masukkan ID Film: 999
- Ekspektasi Output: >> Data berhasil dihapus!

```
=== MENU BIOSKOP ===
1. Tambah Data
2. Tampil Data
3. Edit Data
4. Hapus Data
5. Cari Nama
6. Urutkan ID
7. Urutkan Nama
8. Cari ID
9. Cari Kategori
10. Statistik
11. Urutkan Jumlah Tiket
12. Beli Tiket
0. Keluar
Pilih Menu: 4
Masukkan ID Film: 999
>> Data berhasil dihapus!

142 | Transformers | Action | 45
143 | Cinderella | Fantasy | 15
144 | Annabelle | Horror | 25
145 | Toy Story | Animation | 35
146 | Top Gun | Action | 30
147 | How to Train Your Dragon | Animation | 40
148 | Inside Out | Animation | 50
149 | Mission Impossible | Action | 55
150 | Conjuring | Horror | 20
```

8. Cek Statistik dan Keluar:

- Input: Ketik 10 (Statistik).
- Ekspektasi Output: Total Film Aktif : 50 (karena data ke-51, yaitu The Way Of Water, sudah dihapus).

=== MENU BIOSKOP ===

1. Tambah Data
  2. Tampil Data
  3. Edit Data
  4. Hapus Data
  5. Cari Nama
  6. Urutkan ID
  7. Urutkan Nama
  8. Cari ID
  9. Cari Kategori
  10. Statistik
  11. Urutkan Jumlah Tiket
  12. Beli Tiket
  0. Keluar
- Pilih Menu: 10

=== STATISTIK ===

Total Film Aktif : 50

Film Tersedia : 50

|     |  |                          |  |           |  |    |
|-----|--|--------------------------|--|-----------|--|----|
| 142 |  | Transformers             |  | Action    |  | 45 |
| 143 |  | Cinderella               |  | Fantasy   |  | 15 |
| 144 |  | Annabelle                |  | Horror    |  | 25 |
| 145 |  | Toy Story                |  | Animation |  | 35 |
| 146 |  | Top Gun                  |  | Action    |  | 30 |
| 147 |  | How to Train Your Dragon |  | Animation |  | 40 |
| 148 |  | Inside Out               |  | Animation |  | 50 |
| 149 |  | Mission Impossible       |  | Action    |  | 55 |
| 150 |  | Conjuring                |  | Horror    |  | 20 |

## **BAB 6**

### **Kesimpulan**

Berdasarkan hasil perancangan, implementasi, dan pengujian terhadap kasus studi Sistem Pemesanan Tiket Bioskop, dapat disimpulkan bahwa aplikasi “Sistem Pemesanan Tiket Bioskop” telah berhasil dikembangkan sepenuhnya. Seluruh fungsionalitas utama seperti operasi pengelolaan data (CRUD), transaksi pembelian tiket bioskop, pencarian data, serta pengurutan data mampu berjalan dengan baik sesuai spesifikasi tanpa kendala *error*. Penerapan atribut kontrol aktif terbukti sangat efektif untuk memfasilitasi penghapusan data secara aman (*soft delete*) tanpa merusak struktur sekuensial indeks array. Selain itu, sistem validasi ketersediaan tiket dan penanganan *error* menggunakan blok try-catch berhasil menjaga stabilitas aplikasi dari risiko anomali, seperti masalah *overbooking* maupun kesalahan format *input* dari pengguna.

Dari segi performa algoritma, implementasi *Binary Search* memberikan waktu pencarian ID yang sangat optimal dengan kompleksitas waktu  $O(\log n)$ . Karena pencarian biner mewajibkan data dalam keadaan terurut, sistem juga telah dirancang untuk menjalankan *Bubble Sort* terlebih dahulu secara otomatis guna memastikan akurasi hasil pencarian. Sebagai rekomendasi untuk pengembangan jangka panjang, kapasitas penyimpanan yang saat ini masih menggunakan array statis terbatas 100 elemen dapat dimigrasikan menggunakan struktur data dinamis seperti *ArrayList*. Kinerja sistem juga dapat lebih dioptimalkan dengan mengganti algoritma pengurutan konvensional berordo kuadratik ( $O(n^2)$ ) menjadi algoritma yang lebih efisien untuk skala data besar, seperti *Merge Sort* atau *Quick Sort*.